

Alur Logika Endpoint — POST /api/cv/analyze

Dokumen ini bongkar step-by-step apa yang harus terjadi di dalam controller/service,urut sesuai eksekusi. Anggap tiap step sebagai method terpisah biar testable.

Step 0 — Rate Limit Check

Sebelum apapun diproses.

- Ambil identifier (IP kalau guest, userId kalau login)
- Cek ke Bucket4j: masih ada token/kuota?
- Kalau habis → return 429 Too Many Requests , stop di sini. Jangan lanjut ke parsing (biar gak buang resource).

Step 1 — Validasi Input

- File ada? Tipe file .pdf atau .docx aja? Kalau bukan → 400 Bad Request ("format file tidak didukung")
- Size file wajar (misal max 5MB)? Kalau kelewat → 400
- JD (text) tidak kosong dan punya panjang minimal (misal >50 karakter, biar gak asal paste 1 kata)? Kalau kosong → 400

- Kalau semua valid → lanjut Step 2

Step 2 — Extract Text dari File

- Branch berdasarkan tipe file:
 - `.pdf` → pakai Apache PDFBox, extract raw text
 - `.docx` → pakai Apache POI, extract raw text
- **Penting:** simpan juga info struktural mentah (bukan cuma text polos), karena Gate 1 butuh tahu:
 - Apakah ada multi-column (bisa dicek dari posisi koordinat teks di PDFBox)
 - Apakah ada tabel (POI/PDFBox punya API buat deteksi elemen tabel)
 - Font yang dipakai (PDFBox bisa baca font info per teks run)
- Kalau extract gagal total (file corrupt/kosong) → `422 Unprocessable Entity` ("gagal membaca isi CV")
- Kalau berhasil → lanjut Step 3

Step 3 — Gate 1: Parsing Check (Rule Engine, TANPA AI)

Jalankan serangkaian fungsi cek independen, masing-masing return `pass/fail + detail`:

```
checkLayout(structuralInfo)      → flag kalau
multi-column/tabel/text-box terdeteksi
checkHeadings(rawText, config)  → cocokkan
heading terdeteksi vs headingWhitelist di config
checkFont(structuralInfo, config) → cocokkan font
vs fontWhitelist di config
```

```
checkContactPosition(structuralInfo) → flag kalau  
contact info ada di header/footer area  
checkFileIntegrity(rawText) → flag kalau  
text hasil extract terlalu pendek/aneh (indikasi CV  
berbasis gambar)
```

- Kumpulkan semua hasil jadi `List<ParsingIssue>`
- Hitung `parsingScore` dari jumlah/severity issue (formula ada di config, misal tiap issue kurangi bobot tertentu dari 100)
- Ini semua **sinkron, cepat, tanpa network call** — jadi jalanin duluan sebelum manggil AI (hemat biaya kalau ternyata CV-nya udah parah dari sisi parsing)

Step 4 — Build Prompt untuk Gate 2 & 3 (AI Call)

- Ambil rules dari config (keyword density threshold, dll) → suntik ke system prompt secara eksplisit
- System prompt harus instruksikan AI untuk:
 1. Extract keyword wajib dari JD
 2. Compare ke isi CV (literal matching, bukan asumsi sinonim otomatis)
 3. Nilai posisi tiap keyword (di bullet pencapaian vs cuma disebut)
 4. Cek density tiap keyword penting
 5. Cek kelengkapan (years of experience, pendidikan, sertifikasi vs requirement JD)
 6. Untuk bullet yang lemah, kasih rewrite pakai formula Action verb + keyword + hasil terukur
 7. Wajib output JSON sesuai schema yang ditentukan (lihat Step 5) — jangan free text

- **User message:** kirim `rawText` CV + `jdText`
- Panggil LLM API (model murah dulu untuk MVP, misal Haiku)

Step 5 — Parse & Validasi Response AI

- Response harus JSON dengan schema tetap, contoh:

```
{
  "keywordScore": 0-100,
  "completenessScore": 0-100,
  "matchedKeywords": [...],
  "missingKeywords": [...],
  "keywordIssues": [{"keyword": "...", "issue": "...", "severity": "..."}],
  "rewriteSuggestions": [{"original": "...", "suggestion": "...", "reason": "..."}]
}
```

- **Wajib validasi:** parse JSON di try-catch. Kalau LLM balikin format gak sesuai (kadang kejadian) → retry sekali dengan prompt lebih tegas, atau fallback ke error message yang jelas ke user ("gagal menganalisa, coba lagi")
- Jangan pernah trust output AI 100% mentah — validasi tipe data tiap field sebelum dipakai hitung skor final

Step 6 — Hitung Skor Final

- Ambil `parsingScore` (dari Step 3), `keywordScore` + `completenessScore` (dari Step 5)
- Terapkan formula weighted dari config:

```
totalScore = (parsingScore * weightParsing) +  
(keywordScore * weightKeyword) +  
(completenessScore * weightCompleteness)
```

- Gabungkan semua issue (parsing + keyword) jadi satu list terurut by severity

Step 7 — Generate PDF Report

- Susun data hasil (skor, issue, rewrite suggestion) ke template PDF pakai iText/OpenPDF
- Simpan PDF sementara (misal di storage/temp folder atau langsung stream ke response tanpa simpan permanen kalau guest)

Step 8 — Simpan Hasil (kondisional)

- Kalau user login → simpan `AnalysisResult` ke DB (history)
- Kalau guest → gak perlu simpan permanen, cukup return hasil + PDF langsung (atau simpan dengan TTL pendek buat keperluan download link)

Step 9 — Return Response

- Return JSON: skor breakdown + issues + rewriteSuggestions + link/stream download PDF
 - Status `200 OK`
-

Ringkasan Urutan (checklist implementasi)

1. Rate limit check
2. Validasi input (file + JD)
3. Extract text + structural info
4. Gate 1: rule-based parsing check (tanpa AI) → hitung parsingScore
5. Build system prompt (inject rules dari config) + call LLM
6. Parse & validasi JSON response dari LLM
7. Hitung totalScore (weighted formula)
8. Generate PDF report
9. Simpan hasil kalau user login
10. Return response ke frontend

Prinsip kunci yang harus dipegang: Step 3 (Gate 1) selalu jalan duluan dan gak butuh AI — kalau nanti mau hemat biaya lebih jauh, bisa tambahin logic "kalau parsingScore di bawah threshold tertentu, tetap lanjut ke AI call tapi kasih warning tambahan ke user bahwa parsing jelek akan mempengaruhi hasil real ATS meskipun keyword-nya bagus."